Création d'un chatbot en équipe à l'aide de Git

[A1] Lily AUBERT, Maelys EL ARAARI, Baptiste MASSON (TD2)

Mars 2026

1 Introduction

Pour réaliser ce rendu, nous avons choisi de réaliser un projet fictif, mais qui reste relié à l'une de nos matières de ce semestre : "Introduction à l'IA". Nous avons donc pris le 2e TD que nous avons eu dans cette matière pour l'utiliser en tant que base de ce faux projet. Le but du TD était de créer un petit chatbot, capable d'agir en fonction de règles faites avec des expressions régulières. Notre faux projet est donc le suivant : Créer un petit chatbot et faire une brève introduction sur l'histoire de l'IA pour pouvoir faire une séance d'initiation à l'IA dans un collège. Le projet sera accompagné d'un site web. [A3] Au niveau de la répartition des tâches : Lily à l'écriture du document sur l'histoire de l'IA, Baptiste à la programmation du code en python et Maelys à la rédaction du compte rendu final et pour le site web, tout le monde pouvait contribuer.

1.1 La mise en place

[A2] Pour commencer, nous avons mis en place le dépôt distant git et fait notre premier commit en utilisant la fonctionnalité de GitHub pour ajouter des fichiers par upload (le premier fichier était un simple PDF du début de ce compte rendu). (Lien du dépôt GitHub : `https://GitHub.com/Maelys-EL/ChatBoot`)

Pour réaliser ce projet, nous avons d'ailleurs essayé d'alterner entre les fonctionnalités en ligne de GitHub et les commandes apprises en cours pour nous permettre d'apprendre et d'être à l'aise avec les deux méthodes. Nous verrons donc dans ce compte rendu le parallèle entre les deux.

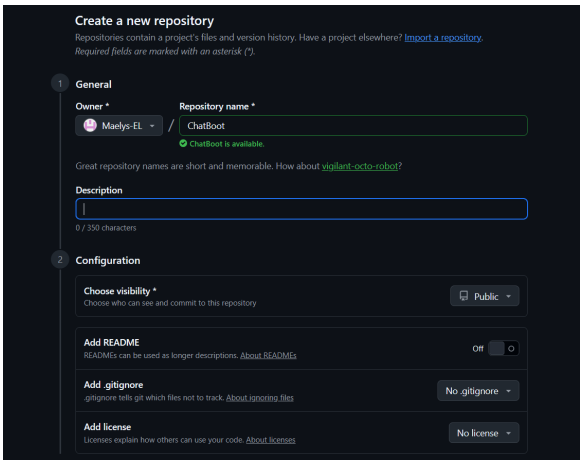


Figure 1: Création du dépôt GitHub

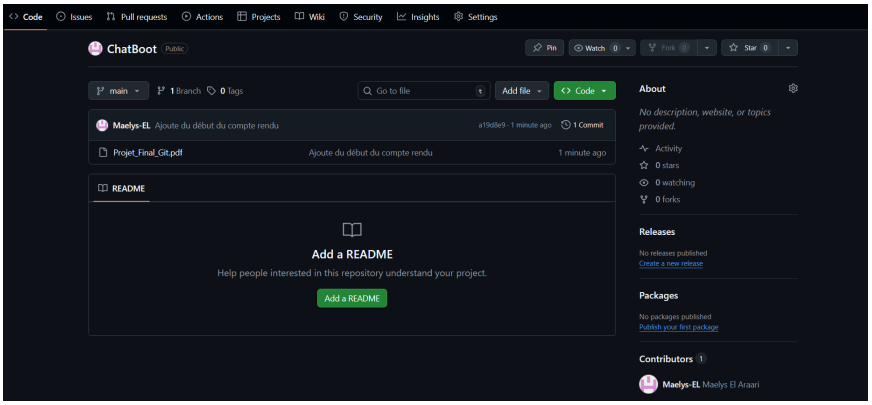
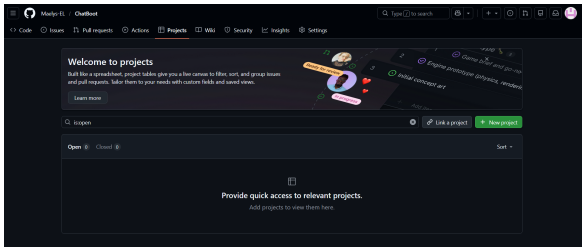


Figure 2: Aperçu du dépôt GitHub après le premier commit

[C4] Après avoir mis en place le dépôt distant, il fallait nous organiser pour créer et répartir les tâches. Comme dit précédemment, des rôles ont été attribués à chacun, mais tout le monde pouvait modifier les fichiers si besoin. Nous avons donc utilisé l'outil "Projet" de GitHub. Pour créer un projet sur GitHub, il faut cliquer sur l'onglet "Projects" puis créer un nouveau projet en cliquant sur le bouton "New project" et surtout faire attention à relier le projet au dépôt correspondant à notre projet pour que les issues soient partagées.



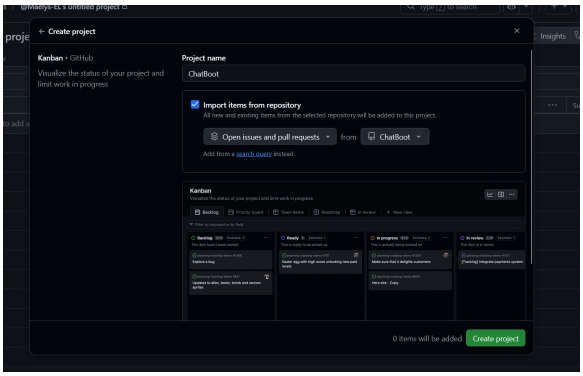


Figure 3: Création du projet sur GitHub

Après la création, il ne faut pas oublier de partager l'accès aux membres du groupe en allant dans les paramètres du projet et dans l'onglet "Collaborators".

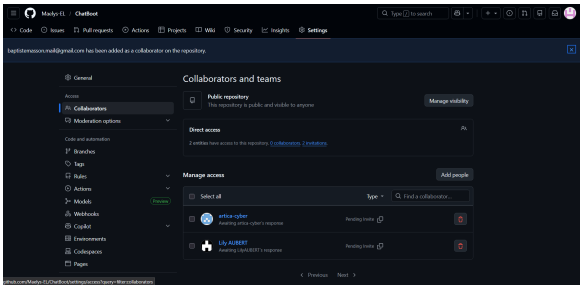


Figure 4: Aperçu de l'ajout de membres

Puis, on peut personnaliser les catégories en les modifiants.

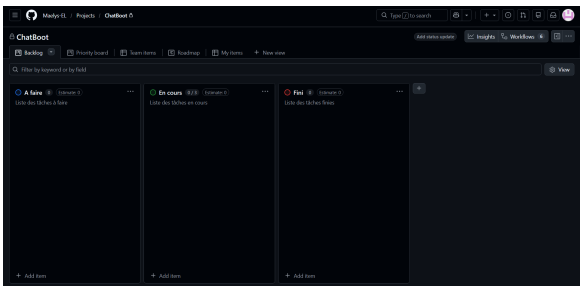


Figure 5: Aperçu du tableau projet modifié

Ce tableau fonctionne de la manière suivante : on crée des tâches (des issues) et on les place simplement dans la colonne correspondante pour visualiser l'avancement du projet. Pour créer les différentes issues, nous avons décidé de créer des labels pour les différentes parties du projet : "Python" pour le code, "Compte rendu", "Site web" et "Histoire de l'IA". À chaque nouvelle

issue, le label correspondant était assigné. Pour créer un label, il faut aller dans l'onglet "issue" sur GitHub, cliquer sur le bouton labels et puis sur le bouton "New label".

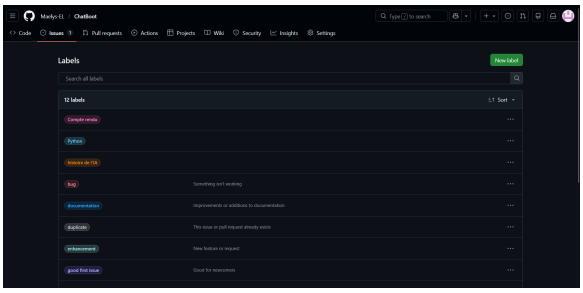
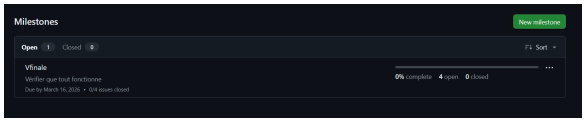


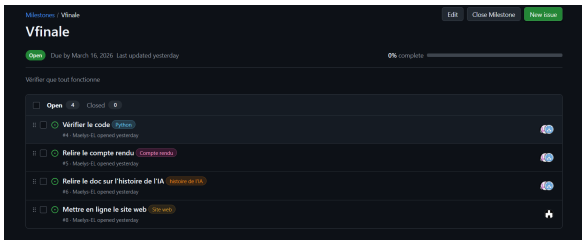
Figure 6: Différents labels

Pour créer des issues, il suffit d'aller dans l'onglet "Issues" sur GitHub, puis cliquer sur le bouton "New issue". On donne ensuite un titre, on assigne la tâche à un ou plusieurs membres du groupe et on utilise les labels créés plutôt. On peut aussi le faire via le tableau du projet créé plutôt.

[C1] Étant donné les délais de réalisation de ce projet, nous n'avons pas pu créer plusieurs milestones, mais nous en avons fait une pour la version finale à rendre le lundi 16/03/2026. Pour créer une milestone, il suffit d'aller dans l'onglet "Issues" de GitHub puis cliquer sur le bouton "Milestones" et enfin cliquer sur le bouton "New milestone".



Les issues créées pour cette milestone sont les tâches de test, vérification, de relecture des documents et mise en ligne du site web.



2 Réalisation du projet

[B1] Pour le projet, nous avons décidé de faire une branche pour chaque partie du projet. La branche main pour le code Python et le document sur l'histoire de l'IA, la branche "Compte_rendu" pour ce document et la branche "Site" pour le site web. Nous avons commencé par créer la branche "Compte_rendu" en ligne de commande. On commence par récupérer le dépôt distant avec la commande :

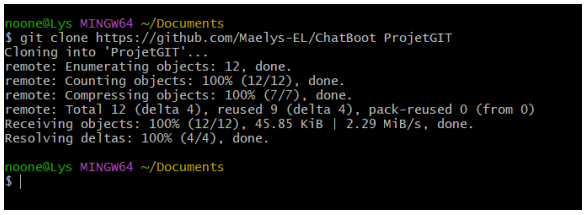
```
git clone [URL du dépôt]
```

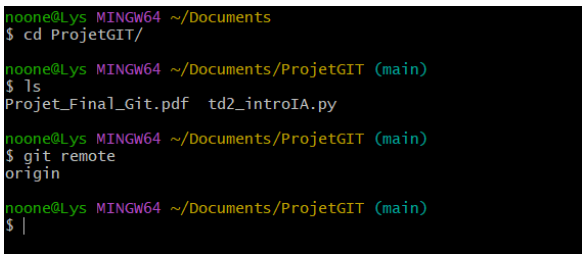
Puis on se déplace sur le premier commit pour créer la branche à partir de là, on utilise donc les commandes :

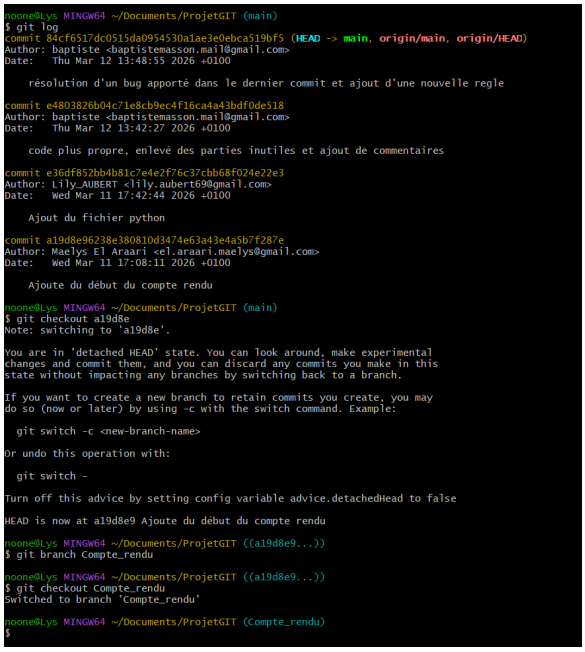
```
git checkout [SHA-1 du 1er commit]
git branch Compte_rendu
```

Enfin, on se déplace sur la branche avec la commande :

```
git checkout Compte_rendu
```





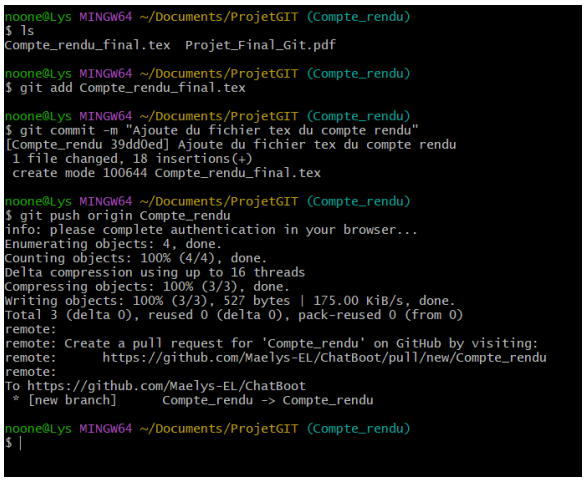


Dans l'exemple ci-dessous, le fichier .tex du compte rendu devait être dépôt, donc on utilise les commandes :

```
git add [nom-du-fichier]
git commit -m "Ajoute du fichier Latex"
```

On finit par push la branche avec la commande git

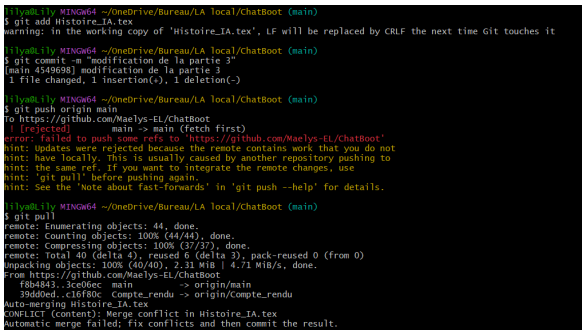
```
git push origin Compte_rendu
```



Pour la branche Site, nous l'avons créée sur GitHub. Ici, il suffit de cliquer sur la branche actuelle main et cliquer sur l'option "View all branches". Puis, on clique sur le bouton "New branches".

[B3] Au cours de notre projet, nous avons dû faire face à un conflit. Un des membres du groupe a modifié un document sans en parler aux autres, un conflit a donc été créé au moment du push sur le dépôt distant. Pour le résoudre, il a fallu modifier le fichier, c'est-à-dire effacer les caractères spéciaux et choisir la version voulue. Puis mettre le tout à jour avec un commit et enfin sur le dépôt distant avec les commandes :

```
git add [nom-du-fichier]
git commit -m "messages"
git push origin main
```



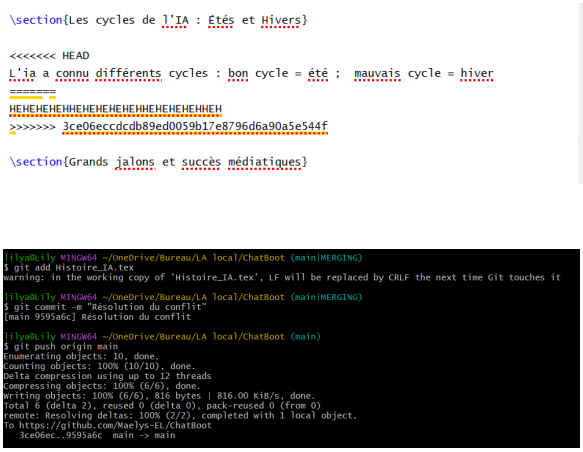


Figure 7: Résolution du conflit

[B3] Après tout ça, il est temps de faire un pull request. Sur la branche "Site" les fichiers "histoire ia.tex" et "td2 introIA.py" ce sont ajoutés. Il faut les supprimer, faire un commit puis faire un pull request via Github pour demander à créer le site. On procède comme suit :

```
git fetch --all (permet de recuperer les branches du dépôt distant)
git branch -a
git checkout Site
rm *
git add .
git commit -m "Suppression des fichiers de la branche site"
git push origin Site
```

```
lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (main)
$ git fetch --all

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (main)
$ git branch -a
  Site
* main
  remotes/origin/Compte_rendu
  remotes/origin/HEAD -> origin/main
  remotes/origin/Site
  remotes/origin/main

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (main)
$ git checkout Site
Switched to branch 'Site'
Your branch is up to date with 'origin/Site'.

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (Site)
$ ls
Histoire_IA.tex  td2_introIA.py

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (Site)
$ rm *

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (Site)
$ ls

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (Site)
$ git add .

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (Site)
$ git commit -m "Suppression des fichiers de la branche Site"
[Site 494b5a6] Suppression des fichiers de la branche Site
 2 files changed, 135 deletions(-)
 delete mode 100644 Histoire_IA.tex
 delete mode 100644 td2_introIA.py

lilya@Lily MINGW64 ~/Onedrive/Bureau/LA local/Chatboot (Site)
$ git push origin Site
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 227 bytes | 227.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Maelys-EL/ChatBoot
   39d9f64..494b5a6  Site -> Site
```

Figure 8: Branche et suppression de fichiers

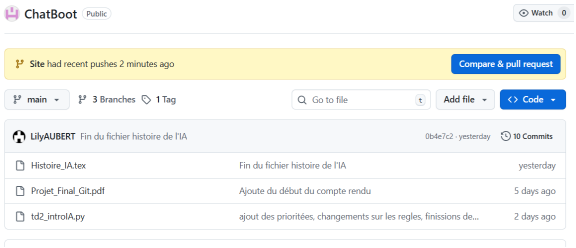
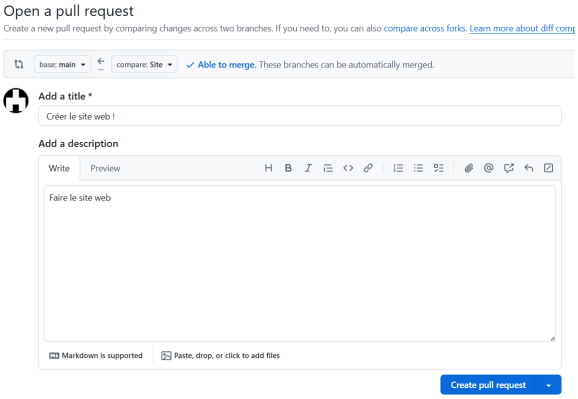


Figure 9: Création d'un pull request



3 Finalisation

[B4] Après avoir créé la branche pour le site web et déposé les fichiers correspondants, il fallait le mettre en ligne. Pour cela, nous avons utilisé les fonctionnalités de GitHub. Pour créer un site sur GitHub, on doit aller sur l'onglet "Settings" puis cliquer sur l'onglet "Pages" et choisir la branche dans laquelle se trouvent les fichiers pour le site web. Finalement, on clique sur le bouton "Save" pour le publier. (Lien du site : `https://maelys-el.github.io/ChatBoot/`)

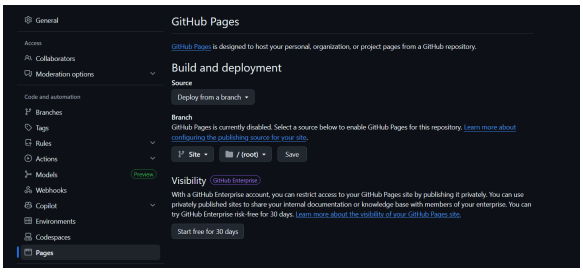


Figure 10: Création du site

Figure 11: Page du site web en ligne

[B2] Comme dit précédemment, nous avons choisi de faire une seule version du projet : la

version finale. Il n'y a pas eu de version intermédiaire donc nous avons seulement créé un tag pour les commits finaux. Pour créer un tag, nous avons fait comme suit :

```
git log (pour récuperer le nom du commit sur le quel mettre le tag)
git tag -a vFinal [numéro du commit]

(Ouvre un éditeur de texte dans le quel écrire le message du tag, ':w'
pour sauvegarder et entrée pour quitter)
git show vFinal (pour vérifier sur quoi porte le tag)
git push origin vFinal
```

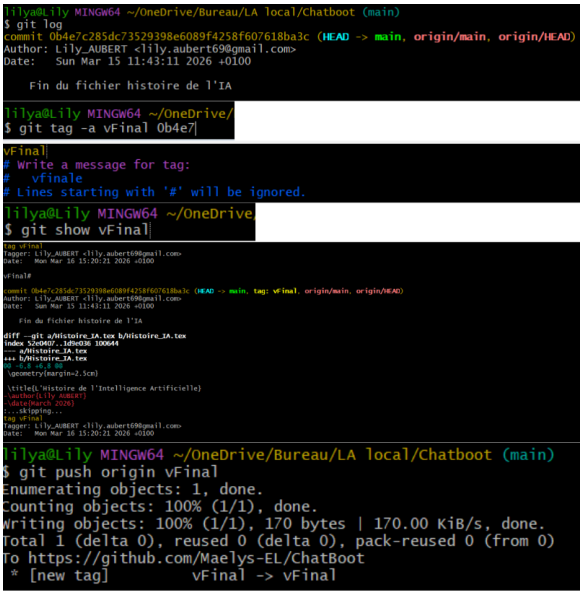


Figure 12: Création d'un tag

Pour finir, nous avons publié la version finale en créant une release sur GitHub.

[C3] Pour finir, il nous fallait mettre en place les submodules. Nous avons seulement utilisé la bibliothèque pandas dans notre code donc nous avons rajouter un seul submodule au projet avec ces commandes :

```
git submodule add https://github.com/pandas-dev/pandas
git add .gitmodules pandas/
git commit -m "Ajoute du submodule"
git push origin main
```

Ici on ajoute le submodule puis on l'ajoute dans l'index avec le dossier cache .gitmodules. Puis on fait un commit et on met à jour le dépôt distant en faisant bien attention à ce que le dépôt local soit mis à jour avec le dépôt distant.

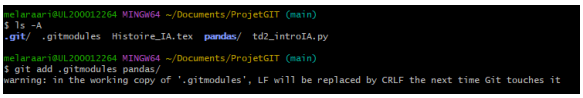


Figure 13: Etat du dépôt après la 1er commande

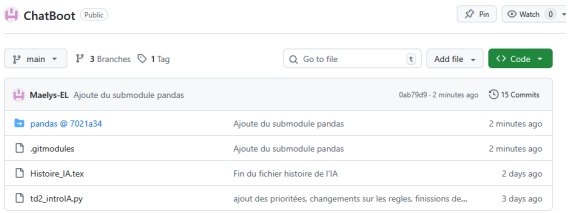


Figure 14: Submodule sur GitHub

4 Conclusion

Avec ce projet, nous avons pu mettre en place les connaissances vues en cours dans un contexte concret et appris à utiliser GitHub. Le projet nous a aussi permis de travailler en groupe pour une durée limitée. Le projet nous a aussi donné envie d'utiliser Git et GitHub plus au quotidien. Lily dit vouloir contribuer à des projets sur GitHub dans le futur et Maelys et Baptiste disent vouloir utiliser Git pour leur projet personnel et/ou universitaire.